Semester Master Security Assurance Manual

Document ID: SEC-MASTER-001
Version: 0.1 draft
Issued: 2026-10-06
Status: Proposed operational standard; implementation and approval pending
Owner: Security lead (appoint named individual)
Approver: Engineering lead and executive risk owner (appoint)
Review: Quarterly, after incidents, and after material architecture changes

1. Purpose and assurance limitations

Define repeatable preventive, detective, corrective, and recovery controls for Semester. This package is a proposed baseline, not a completed audit, penetration test, certification, or legal compliance determination. Existing documents and controls must be verified against the current commit and deployed environments. All CSV controls start NOT_VERIFIED. Absence of findings is not proof of security.

The working architecture is a React/Vite/TypeScript application with Supabase authorization and Vercel deployment. Verify this inventory before testing. The previously discussed permission model includes app_roles, role_capabilities, role_grants, and private.has_capability; confirm actual signatures and behavior before implementation.

2. Scope and inventory

Include the student application, company website, admin console, APIs, database schemas, storage, realtime, edge/server functions, background jobs, workflow coordinators, integrations, GitHub workflows, deployment settings, DNS, backups, and operational accounts. Record third parties and explicit exclusions. For every component record owner, repository path, deployment/project ID, environment, exposure, data classification, authentication method, dependency, backup policy, and evidence location.

Environments: isolated local/test, staging, and production. Use synthetic fixtures by default. Production data cannot enter lower environments without documented authorization and an approved minimization/redaction process. Never assume staging is private because its URL is obscure.

3. Governance and responsibilities

Engineering owns implementation, CI gates, migrations, and recovery. Security owns verification, threat models, findings, and exceptions. Operations owns monitoring and incident execution. Privacy/legal owns legal applicability, agreements, and notification decisions. Product owns user safety and user-facing documentation. Assign named primary and backup people; small teams may combine roles but need independent approval for high-risk changes.

Every control has an accountable owner, scope, method, acceptance criterion, evidence, due date, and review date. Statuses: NOT_VERIFIED, PASS, FAIL, NOT_APPLICABLE, ACCEPTED_EXCEPTION. PASS requires current scoped evidence. NOT_APPLICABLE needs an approved rationale. Exceptions require risk owner, compensating controls, expiry, and a tracked retest. A score never overrides a failed launch blocker.

4. Standards and mapping policy

Use OWASP ASVS as the application verification catalog and NIST SSDF as the secure-development lifecycle framework. The attached matrix deliberately maps to ASVS 4.0.3 CHAPTERS, not individual requirements. ASVS 5.0.0 is current; chapter numbers changed, so do not relabel this matrix as 5.0.0. Before an external ASVS assessment, import the official versioned requirements, map every applicable requirement, validate IDs and levels, identify gaps, and retain the assessment scope. Target Level 2 coverage subject to validated requirement-level mapping. This starter matrix is neither exhaustive ASVS coverage nor a compliance attestation.

5. Data and threat modeling

Classify data as public, internal, confidential, or restricted. Student records, identity credentials, access grants, financial information, and sensitive communications require explicit handling decisions. Record source, purpose, legal basis/authority as applicable, retention, access, export rules, deletion, backup retention, and subprocessors.

Threat models must cover spoofing, tampering, repudiation, disclosure, denial of service, and privilege escalation. Include cross-institution access, malicious insiders, compromised integrations, account recovery abuse, secret leakage, imports, exports, and stale policy decisions. High-impact workflows need fail-safe defaults, fresh authorization, bounded authority, idempotency, reconciliation, and audited compensation. AI features, if present, need prompt-injection tests, bounded tools, output validation, and human approval for consequential writes.

6. Control operation

Use control-matrix.csv as the starter implementation and evidence register. Inventory all resources before claiming coverage. Each feature must implement server-side authorization, input validation, safe outputs, rate/size limits, audit events, and failure/recovery behavior appropriate to its risk. UI visibility is not authorization. Feature flags are not access controls. Record resource tenant ownership and prevent unauthorized ownership changes.

Apply the accompanying access/RLS standard to tables, views, functions, storage, realtime subscriptions, and privileged workers. Verify API exposure separately from database membership. Review object grants and row policies together.

7. Verification and release gates

Every PR: lint/typecheck, applicable tests, secret scan, dependency review, SAST, policy/migration tests, and reviewer approval. Run scanner updates without executing untrusted contributions with production credentials. High-risk changes require security review. Staging release: authenticated DAST, manual authorization tests, abuse cases, accessibility checks, monitoring verification, and rollback plan. Test only explicitly authorized targets.

Block release for confirmed unauthorized sensitive-data access, privilege escalation, exposed usable secrets, unresolved exploitable critical findings, unsafe migrations, or missing recovery for destructive changes. High findings need remediation or explicit short-lived approval and verified mitigation; data-exposure blockers cannot be waved through by a score. Re-run tests on the exact release candidate. Record commit, build, dependencies, configuration version, migration set, approvals, evidence, deployment, and post-release verification.

8. Monitoring and evidence

Log actor ID, institution/tenant, action, resource reference, allow/deny outcome, reason category, correlation ID, timestamp, and relevant policy/version; minimize identifiers where possible. Never log tokens, passwords, service keys, raw sensitive content, or full student records. Logs need access limits, retention, tamper protection appropriate to risk, and time synchronization. Test alerts for repeated denials, grant changes, unusual exports, suspicious login/recovery, and infrastructure failures.

Evidence records: evidence ID, control/test IDs, scope, commit/deployment, environment, tester, UTC time, expected/actual outcomes, sanitized artifact, artifact hash, retention, and reviewer. Store evidence in access-controlled storage, not publicly readable repository artifacts. Screenshots alone rarely prove enforcement.

9. Remediation and incident management

Finding lifecycle: detect -> validate -> contain if needed -> classify -> assign -> fix -> independent retest -> deploy -> monitor -> close. Proposed targets: critical contain immediately and fix within 24 hours where feasible; high within 7 days; medium within 30; low within 90. These are internal targets, not legal notification deadlines. Escalate missed deadlines and renew exceptions explicitly.

Suspected breach: appoint incident commander, preserve evidence, restrict affected access, rotate compromised credentials, investigate scope, recover, and involve privacy/legal for notice obligations. Do not delay containment for a perfect root cause. Postmortems document timeline, impact, contributing causes, controls added, owners, and due dates. See the testing/debugging runbook for detailed steps.

10. Recovery and continuity

Approve RPO/RTO by data/workflow before launch; do not assume provider defaults meet institutional needs. Verify backup coverage of database, files, configuration, integration state, and keys. Execute isolated restore exercises at least quarterly under this proposed policy, validate authorization and integrity after restore, and measure elapsed recovery/data loss. Deployment rollback does not roll back database migrations; use compatible expand/contract changes and a separate data recovery plan. Coordinate key rotation and recovery so backups cannot reinstate compromised credentials or stale grants.

11. Compliance applicability matrix

Framework/obligation	Applicability question	Evidence/controls	Approval boundary
OWASP ASVS	Which version, level, and requirements are in scope?	Versioned requirements, test evidence, exceptions	Security verifies coverage; no certification claim from this starter
NIST SSDF	Which lifecycle tasks apply?	Governance, source protection, secure builds, testing, vulnerability response	Engineering/security validate task-level mapping
FERPA	Does Semester handle education records under an institution's arrangements?	Institution contract, access/purpose limits, export/deletion procedures, access logs	Institution/privacy counsel determine applicability and contractual terms
GLBA Safeguards Rule	Is an applicable institution or service relationship handling covered information?	Risk assessment, safeguards, vendor oversight, incident procedures	Legal determines scope and duties
GDPR/UK GDPR	Is territorial/data-processing scope triggered?	Data inventory, contracts, rights workflow, retention, transfers	Privacy counsel determines roles and obligations
State privacy/breach laws	Which jurisdictions and data categories are involved?	Data inventory, response process, notice decision record	Counsel determines exemptions and deadlines
COPPA	Is the service directed to children under 13 or knowingly collecting their data?	Age/audience review, approved consent/data handling where applicable	Legal/product determine scope; do not assume college-only use
PCI DSS	Does any system store/process/transmit payment card data or affect that environment?	Payment architecture, provider evidence, scope review	Qualified payment/compliance review
SOC 2/ISO 27001	Is formal organizational assurance sought or contractually required?	Defined organizational scope and independent assessment evidence	No claim of attestation/certification without it
Accessibility	What contractual and legal accessibility requirements apply?	WCAG target, automated/manual assistive-technology tests	Product/legal validate commitments

This table is a legal applicability intake, not a legal opinion or proof of compliance. Map actual contracts, jurisdictions, and data flows before making claims.

12. Manuals and review cadence

User manual: account safety, recovery, privacy settings, data provenance/freshness, export/deletion requests, suspicious activity reporting, and irreversible-action warnings. Admin manual: scoped role changes, approvals, revocations, audit review, exports, account suspension, and break-glass limits. Operator manual: triage, deployments, migration checks, credential rotation, rollback, restore, and escalation. Keep sensitive operator procedures access-controlled.

Proposed cadence: per PR automated checks; weekly triage and drift review; monthly dependency/access/alert review; quarterly privilege review, restore test, and control reassessment; annual or major-change independent penetration test scoped to risk. Owners must approve the schedule and resource it.

13. Initial execution checklist

[ ] Confirm architecture and asset inventory against current production.
[ ] Name owners/approvers and authorize assessment scope.
[ ] Inventory exposed objects, policies, grants, definer functions, and privileged jobs.
[ ] Execute negative cross-tenant authorization tests.
[ ] Validate CI gates and scanner coverage.
[ ] Complete threat models for high-impact flows.
[ ] Validate runbooks through tabletop and restore exercises.
[ ] Assess legal/contractual applicability and register exceptions.
[ ] Replace chapter-level ASVS mapping with validated current-version requirements.
[ ] Approve a bounded release only after launch blockers are closed with evidence.

Sources

OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/
ASVS versioned releases: https://github.com/OWASP/ASVS/releases
NIST SSDF: https://csrc.nist.gov/projects/ssdf
Supabase RLS: https://supabase.com/docs/guides/database/postgres/row-level-security
Supabase API security: https://supabase.com/docs/guides/api/securing-your-api
Supabase functions: https://supabase.com/docs/guides/database/functions